

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

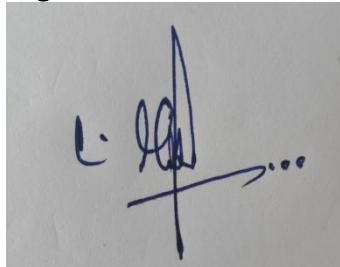
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I Mukesh Raj (192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

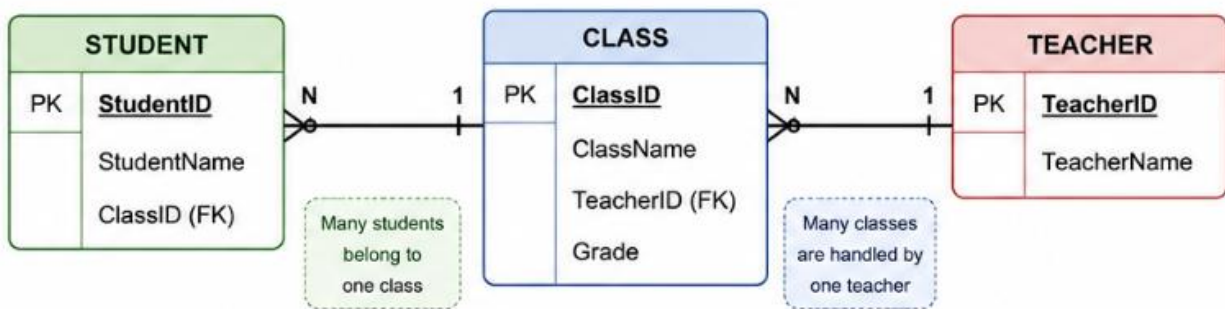
1)

Scenario

A **School Management System** stores information about:

- Students
- Teachers
- Classes
- Grades

Each **student belongs to one class**, and **each class has one class teacher**.



Step 1: Identify the Relations (Tables)

1. STUDENT

Attribute	Description
StudentID (PK)	Unique Student ID
StudentName	Student Name
ClassID (FK)	Class in which student studies

2. CLASS

Attribute	Description
ClassID (PK)	Unique Class ID
ClassName	Class Name
TeacherID (FK)	Class Teacher ID
Grade	Student Grade

3. TEACHER

Attribute	Description
TeacherID (PK)	Unique Teacher ID
TeacherName	Teacher Name

Step 2: Primary Keys and Foreign Keys

Primary Keys

- StudentID → STUDENT

- ClassID → CLASS
- TeacherID → TEACHER

Foreign Keys

- ClassID in STUDENT references CLASS(ClassID)
- TeacherID in CLASS references TEACHER(TeacherID)

Step 3: SQL Query

```
SELECT
    S.StudentID,
    S.StudentName,
    C.ClassName,
    T.TeacherName AS ClassTeacher,
    C.Grade
FROM STUDENT S
INNER JOIN CLASS C
ON S.ClassID = C.ClassID
INNER JOIN TEACHER T
ON C.TeacherID = T.TeacherID;
```

Explanation

- SELECT retrieves the required details.
- INNER JOIN CLASS matches each student with their class.
- INNER JOIN TEACHER matches each class with its class teacher.
- The query displays:
 - Student ID
 - Student Name
 - Class Name
 - Class Teacher
 - Grade

Sample Tables

STUDENT

StudentID	StudentName	ClassID
101	Arun	C1
102	Bala	C2
103	Charan	C1

CLASS

ClassID	ClassName	TeacherID	Grade
C1	X-A	T1	A
C2	X-B	T2	B

TEACHER

TeacherID	TeacherName
-----------	-------------

TeacherID	TeacherName
T1	Mr. Kumar
T2	Ms. Priya

Expected Output

StudentID	StudentName	ClassName	ClassTeacher	Grade
101	Arun	X-A	Mr. Kumar	A
102	Bala	X-B	Ms. Priya	B
103	Charan	X-A	Mr. Kumar	A

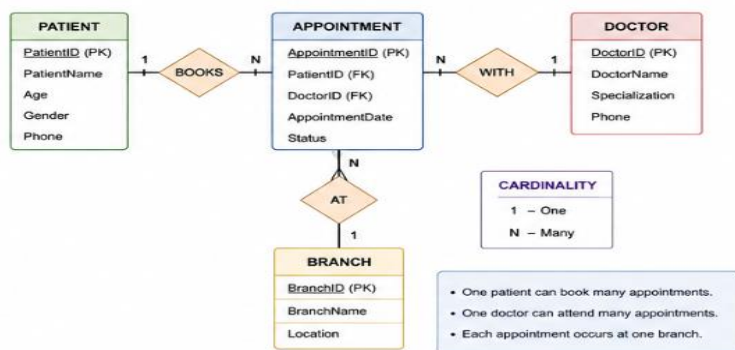
2)

Scenario

A **Hospital Management System** maintains records of:

- Patients
- Doctors
- Appointments

Each **patient** can have many appointments, and each **doctor** can attend many appointments.



Step 1: Relational Schema

1. PATIENT

Attribute	Description
PatientID (PK)	Unique Patient ID
PatientName	Patient Name
Age	Patient Age
Gender	Patient Gender

2. DOCTOR

Attribute	Description
-----------	-------------

Attribute	Description
DoctorID (PK)	Unique Doctor ID
DoctorName	Doctor Name
Specialization	Doctor's Specialization

3. APPOINTMENT

Attribute	Description
AppointmentID (PK)	Unique Appointment ID
PatientID (FK)	References PATIENT
DoctorID (FK)	References DOCTOR
AppointmentDate	Date of Appointment

Step 2: Primary Keys and Foreign Keys

Primary Keys

- PatientID → PATIENT
- DoctorID → DOCTOR
- AppointmentID → APPOINTMENT

Foreign Keys

- PatientID in APPOINTMENT references PATIENT(PatientID)
- DoctorID in APPOINTMENT references DOCTOR(DoctorID)

Step 3: SQL Query

The following query lists doctors who have **more than 10 appointments in the current month**.

```
SELECT
    D.DoctorID,
    D.DoctorName,
    COUNT(A.AppointmentID) AS TotalAppointments
FROM DOCTOR D
INNER JOIN APPOINTMENT A
ON D.DoctorID = A.DoctorID
WHERE MONTH(A.AppointmentDate) = MONTH(CURRENT_DATE)
AND YEAR(A.AppointmentDate) = YEAR(CURRENT_DATE)
GROUP BY D.DoctorID, D.DoctorName
HAVING COUNT(A.AppointmentID) > 10;
```

Explanation

- INNER JOIN combines the **DOCTOR** and **APPOINTMENT** tables.
- WHERE filters appointments to the **current month**.
- COUNT(AppointmentID) counts the number of appointments for each doctor.
- GROUP BY groups the records by doctor.
- HAVING COUNT(AppointmentID) > 10 displays only those doctors who have **more than 10 appointments**.

Sample Tables

DOCTOR

DoctorID	DoctorName	Specialization
D101	Dr. Kumar	Cardiology
D102	Dr. Priya	Neurology

APPOINTMENT

AppointmentID	PatientID	DoctorID	AppointmentDate
A001	P101	D101	2026-08-01
A002	P102	D101	2026-08-02
...	...	...	...

If **Dr. Kumar** has **12 appointments** this month, the query returns:

DoctorID	DoctorName	TotalAppointments
D101	Dr. Kumar	12

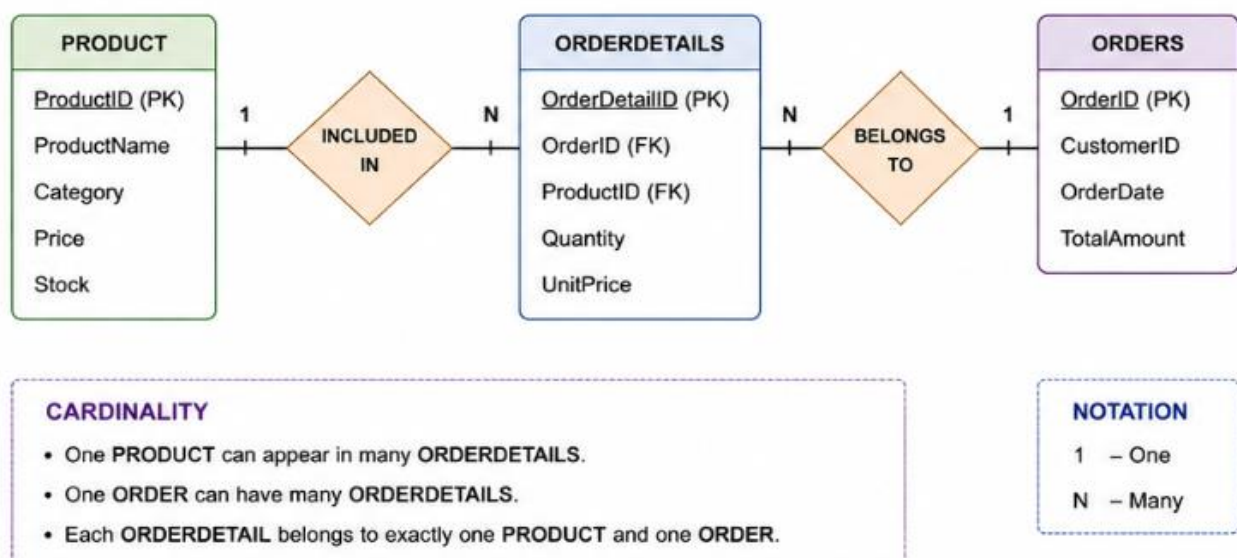
3)

Scenario

An **E-commerce System** stores information about:

- Products
- Customers
- Orders

The task is to **find products that have never been ordered** using a **subquery**.



Step 1: Relational Schema

1. PRODUCT

Attribute	Description
ProductID (PK)	Unique Product ID
ProductName	Product Name
Price	Product Price

2. CUSTOMER

Attribute	Description
CustomerID (PK)	Unique Customer ID
CustomerName	Customer Name

3. ORDERS

Attribute	Description
OrderID (PK)	Unique Order ID
CustomerID (FK)	References CUSTOMER
ProductID (FK)	References PRODUCT
OrderDate	Date of Order

Step 2: SQL Query Using Subquery

```
SELECT ProductID, ProductName FROM PRODUCT WHERE ProductID NOT IN  
(  
    SELECT ProductID  
    FROM ORDERS  
);
```

Explanation

- The **inner query** retrieves all ProductIDs that have been ordered from the **ORDERS** table.
- The **outer query** selects products from the **PRODUCT** table.
- NOT IN removes all products that appear in the **ORDERS** table.
- The result shows **only the products that have never been ordered**.

Sample Tables

PRODUCT

ProductID	ProductName	Price
P101	Laptop	50000
P102	Mouse	800
P103	Keyboard	1200
P104	Printer	7000

ORDERS

OrderID	CustomerID	ProductID
O101	C101	P101
O102	C102	P102

Expected Output

ProductID	ProductName
-----------	-------------

ProductID	ProductName
P103	Keyboard
P104	Printer

These products have **never been ordered**.

4)

Scenario

A **University Database** contains information about:

- Students
- Courses
- Enrollments

The objective is to **find students who are enrolled in both the "DBMS" and "OS" courses**.

Step 1: Relations (Tables)

STUDENT

Attribute	Description
SID (PK)	Student ID
StudentName	Student Name

COURSE

Attribute	Description
CID (PK)	Course ID
CourseName	Course Name

ENROLLMENT

Attribute	Description
SID (FK)	Student ID
CID (FK)	Course ID

Step 2: Relational Algebra Expression

Find students enrolled in DBMS

$R1 = \pi_{SID}(\sigma_{CourseName='DBMS'}(COURSE \bowtie ENROLLMENT))$

Find students enrolled in OS

$R2 = \pi_{SID}(\sigma_{CourseName='OS'}(COURSE \bowtie ENROLLMENT))$

Find students enrolled in both DBMS and OS

$R = R1 \cap R2$

Display student names

$\pi_{StudentName}(STUDENT \bowtie R)$

Complete Relational Algebra Expression

$R1 = \pi_{SID}(\sigma_{CourseName='DBMS'}(COURSE \bowtie ENROLLMENT))$

$R2 = \pi_{SID}(\sigma_{CourseName='OS'}(COURSE \bowtie ENROLLMENT))$

$$R = R1 \cap R2$$

$$\text{Answer} = \pi_{\text{StudentName}}(\text{STUDENT} \bowtie R)$$

Explanation

- σ (Selection) selects the required courses (DBMS and OS).
- $\bowtie$ (Join) combines the COURSE and ENROLLMENT tables.
- π (Projection) retrieves only the SID.
- $\cap$ (Intersection) finds students enrolled in both courses.
- Finally, joining with the STUDENT table retrieves the student names.

Example

STUDENT

SID	StudentName
101	Arun
102	Bala
103	Charan

COURSE

CID	CourseName
C1	DBMS
C2	OS

ENROLLMENT

SID	CID
101	C1
101	C2
102	C1
103	C2

Output

StudentName
Arun

Reason: Arun is enrolled in **both DBMS and OS**.

5)

Scenario

A company maintains employee salary records. The objective is to:

1. Create a **view** that displays employees whose salary is **less than the average salary of their department**.
2. Increase the salary of those employees by **10%**.

Step 1: Relational Schema

EMPLOYEE

Attribute	Description
EmpID (PK)	Employee ID
EmpName	Employee Name
DeptID (FK)	Department ID
Salary	Employee Salary

DEPARTMENT

Attribute	Description
DeptID (PK)	Department ID
DeptName	Department Name

Step 2: Create the View

```
CREATE VIEW BelowDeptAvgSalary AS
SELECT E.EmpID,
       E.EmpName,
       E.DeptID,
       E.Salary
FROM EMPLOYEE E
WHERE E.Salary <
(
  SELECT AVG(E2.Salary)
  FROM EMPLOYEE E2
  WHERE E2.DeptID = E.DeptID
);
```

Explanation

- CREATE VIEW creates a virtual table named **BelowDeptAvgSalary**.
- The subquery calculates the **average salary** for each department.
- Employees whose salary is **less than their department's average salary** are included in the view.

Step 3: Update Salary by 10%

```
UPDATE EMPLOYEE SET Salary = Salary * 1.10 WHERE EmpID IN
(
  SELECT EmpID
  FROM BelowDeptAvgSalary
);
```

Explanation

- UPDATE modifies employee salaries.
- Salary * 1.10 increases the salary by **10%**.
- The WHERE clause ensures that **only employees in the view** are updated.

Example

EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Arun	D1	40000
102	Bala	D1	60000
103	Charan	D2	30000
104	Deepak	D2	50000

Department Average Salary:

- $D1 = (40000 + 60000) / 2 = \mathbf{50000}$
- $D2 = (30000 + 50000) / 2 = \mathbf{40000}$

Employees below average:

- Arun (40000)
- Charan (30000)

After the update:

- Arun $\rightarrow \mathbf{44000}$
- Charan $\rightarrow \mathbf{33000}$

6)

Step 1: Identify the Relations

1. CUSTOMER

CUSTOMER(CustomerID, CustomerName, Phone)

- CustomerID $\rightarrow$ Primary Key

2. ACCOUNT

ACCOUNT(AccountID, CustomerID, BranchID, AccountType, Balance)

- AccountID $\rightarrow$ Primary Key
- CustomerID $\rightarrow$ Foreign Key
- BranchID $\rightarrow$ Foreign Key

3. TRANSACTION

TRANSACTION(TransactionID, AccountID, TransactionDate, TransactionType, Amount)

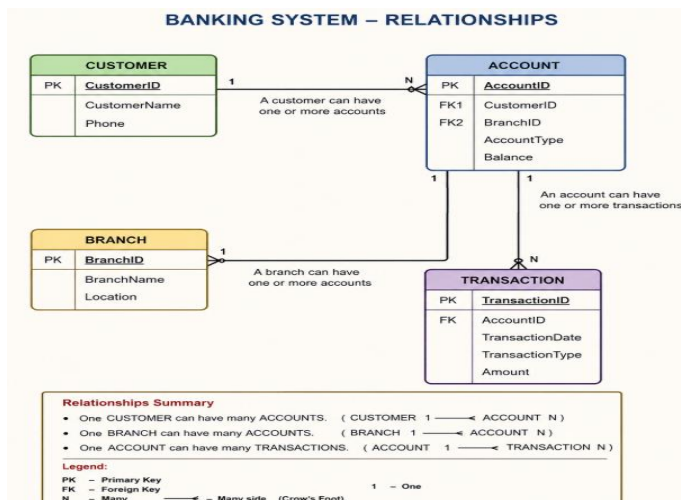
- TransactionID $\rightarrow$ Primary Key
- AccountID $\rightarrow$ Foreign Key

4. BRANCH

BRANCH(BranchID, BranchName, Location)

- BranchID $\rightarrow$ Primary Key

Step 2: Relationships



So, we can use **JOIN operations** to retrieve complete banking information.

Step 3: INNER JOIN

Find customer account details and branch information

SELECT

C.CustomerID,
 C.CustomerName,
 A.AccountID,
 A.AccountType,
 A.Balance,
 B.BranchName,

B.Location
 FROM CUSTOMER C
 INNER JOIN ACCOUNT A ON C.CustomerID = A.CustomerID
 INNER JOIN BRANCH B ON A.BranchID = B.BranchID;

Explanation

- CUSTOMER is joined with ACCOUNT using CustomerID.
- ACCOUNT is joined with BRANCH using BranchID.
- It displays customers who have accounts along with their account and branch details.

Step 4: JOIN with Transaction History

To retrieve **customer, account, transaction, and branch information**:

SELECT

C.CustomerName,
 A.AccountID,
 A.AccountType,
 T.TransactionID,
 T.TransactionDate,
 T.TransactionType,
 T.Amount,

B.BranchName
 FROM CUSTOMER C
 INNER JOIN ACCOUNT A ON C.CustomerID = A.CustomerID
 INNER JOIN TRANSACTION T ON A.AccountID = T.AccountID
 INNER JOIN BRANCH B ON A.BranchID = B.BranchID;

Explanation

This query combines **four tables**:

CUSTOMER → ACCOUNT → TRANSACTION
↓
BRANCH

It displays:

- Customer name
- Account number
- Account type
- Transaction ID
- Transaction date
- Transaction type
- Transaction amount
- Branch name

Step 5: LEFT JOIN

To display **all customers**, including customers who do not have an account:

```
SELECT  
  C.CustomerName,  
  A.AccountID,  
  A.AccountType,  
  A.Balance  
FROM CUSTOMER C LEFT JOIN ACCOUNT A ON C.CustomerID = A.CustomerID;
```

If a customer does not have an account, the account details will appear as NULL.

Sample Output

CustomerName	AccountID	AccountType	TransactionType	Amount	BranchName
Arun	A101	Savings	Deposit	10000	Main Branch
Bala	A102	Current	Withdrawal	5000	City Branch
Charan	A103	Savings	Deposit	15000	Main Branch

7)

Step 1: Assume the SALES Table

SALES(

SaleID,

ProductName,

Category,

SaleDate,

Amount

)

- SaleID → Unique sale ID

- ProductName → Name of the product
- Category → Product category
- SaleDate → Date of sale
- Amount → Sales amount

Step 2: SQL Query

To find the **total sales for each category** and display only categories with sales **greater than Rs. 10,000**:

```
SELECT
    Category,
    SUM(Amount) AS TotalSales
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 10000;
```

Step 3: For Daily Sales Summary

Since the question specifically says **daily sales summary**, it is better to include the sale date:

```
SELECT
    SaleDate,
    Category,
    SUM(Amount) AS TotalSales
FROM SALES
GROUP BY SaleDate, Category
HAVING SUM(Amount) > 10000;
```

This is the best answer because:

- SUM(Amount) → calculates total sales.
- GROUP BY SaleDate, Category → creates a separate total for each category on each day.
- HAVING SUM(Amount) > 10000 → displays only categories whose total sales exceed Rs. 10,000.

Example SALES

SaleID	ProductName	Category	SaleDate	Amount
1	Laptop	Electronics	2026-08-05	6000
2	Mobile	Electronics	2026-08-05	7000
3	Chair	Furniture	2026-08-05	4000

SaleID	ProductName	Category	SaleDate	Amount
4	Table	Furniture	2026-08-05	3000

Calculation

Electronics:

$6000 + 7000 = \text{Rs. } 13,000$

Furniture:

$4000 + 3000 = \text{Rs. } 7,000$

Because only Electronics exceeds Rs. 10,000:

Output

SaleDate	Category	TotalSales
2026-08-05	Electronics	13000

8)

Step 1: Relations

MEMBER

MEMBER(MemberID, MemberName)

BOOK

BOOK(BookID, BookTitle, GenreID)

GENRE

GENRE(GenreID, GenreName)

BORROW

BORROW(MemberID, BookID, BorrowDate)

Step 2: Understanding the Requirement

We need to find members who have borrowed **at least one book from every genre**.

For each genre in the GENRE relation, there must exist:

1. A book belonging to that genre.
2. A borrow record for that book.
3. The borrow record must belong to the particular member.

Step 3: Tuple Relational Calculus Expression

```
{ M |
  MEMBER(M) AND
  FOR ALL G (
    GENRE(G) →
    EXISTS B (
      BOOK(B) AND
      B.GenreID = G.GenreID AND
```

```

    EXISTS BR (
      BORROW(BR) AND
      BR.MemberID = M.MemberID AND
      BR.BookID = B.BookID
    )
  )
}

```

Explanation

MEMBER(M)

Means M is a tuple from the **MEMBER** relation.

FOR ALL G

Means we check **every genre** in the GENRE table.

GENRE(G) →

For every existing genre, the remaining condition must be satisfied.

EXISTS B

There must be at least one book belonging to that genre.

B.GenreID = G.GenreID

Connects the book with its genre.

EXISTS BR

There must be a borrowing record for that book.

BR.MemberID = M.MemberID

Ensures that the **member actually borrowed the book**.

BR.BookID = B.BookID

Connects the borrowing record with the book.

Therefore, the expression returns members who have borrowed **at least one book from every genre**.

Simple Example

Suppose the library has these genres:

Fiction
Science
History

If **Arun** has borrowed:

Fiction book
Science book
History book

then Arun will be included in the result.

If **Bala** has borrowed only:

Fiction book
Science book

then Bala will **not** be included because Bala has not borrowed a History book.

9)

1. What is a CHECK Constraint?

A **CHECK constraint** ensures that a column contains only values satisfying a specified condition.

For example, product quantity cannot be negative and product price must be greater than zero.

2. What is a FOREIGN KEY Constraint?

A **FOREIGN KEY** creates a relationship between two tables.

It ensures that a value in the child table must exist as a **PRIMARY KEY** in the parent table.

Step 1: Create the CATEGORY Table

```
CREATE TABLE CATEGORY
(
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL
);
```

Here, CategoryID is the **Primary Key**.

Step 2: Create the PRODUCT Table

```
CREATE TABLE PRODUCT
(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    CategoryID INT,
    Quantity INT CHECK (Quantity >= 0),
    Price DECIMAL(10,2) CHECK (Price > 0),

    FOREIGN KEY (CategoryID)
    REFERENCES CATEGORY(CategoryID)
);
```

Constraints used:

CHECK constraints:

Quantity >= 0
Price > 0

These prevent invalid inventory values.

FOREIGN KEY:

FOREIGN KEY (CategoryID)
REFERENCES CATEGORY(CategoryID)

This ensures that the category assigned to a product exists in the CATEGORY table.

Step 3: Insert Valid Data

First insert a category:

```
INSERT INTO CATEGORYVALUES (1, 'Electronics');
```

Now insert a valid product:

```
INSERT INTO PRODUCTVALUES (101, 'Laptop', 1, 20, 50000);
```

This is accepted because:

- Quantity = 20 → valid
- Price = 50000 → valid
- CategoryID = 1 exists in CATEGORY

Step 4: Demonstrate CHECK Constraint

Try inserting a product with negative quantity:

```
INSERT INTO PRODUCTVALUES (102, 'Mouse', 1, -5, 800);
```

Result:

The database **rejects the insertion** because:

Quantity >= 0

is violated.

Similarly:

```
INSERT INTO PRODUCTVALUES (103, 'Keyboard', 1, 10, -100);
```

This is rejected because:

Price > 0

is violated.

Step 5: Demonstrate FOREIGN KEY Constraint

Try inserting a product with a category that does not exist:

```
INSERT INTO PRODUCTVALUES (104, 'Printer', 10, 5, 7000);
```

Here:

CategoryID = 10

does not exist in the CATEGORY table.

Therefore, the database **rejects the insertion** due to the **FOREIGN KEY constraint**.

10)

1. Relational Schema for HR System

We can use four main relations:

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID, Designation)

- EmpID → Primary Key
- DeptID → Foreign Key

DEPARTMENT

DEPARTMENT(DeptID, DeptName)

- DeptID → Primary Key

ATTENDANCE

ATTENDANCE(AttendanceID, EmpID, AttendanceDate, Status)

- AttendanceID → Primary Key
- EmpID → Foreign Key
- Status → Present / Absent

PERFORMANCE

PERFORMANCE(PerformanceID, EmpID, ReviewDate, Rating)

- PerformanceID → Primary Key
- EmpID → Foreign Key
- Rating → Employee performance rating

2. Create the Tables

DEPARTMENT

CREATE TABLE DEPARTMENT

(

DeptID INT PRIMARY KEY,
DeptName VARCHAR(50) NOT NULL

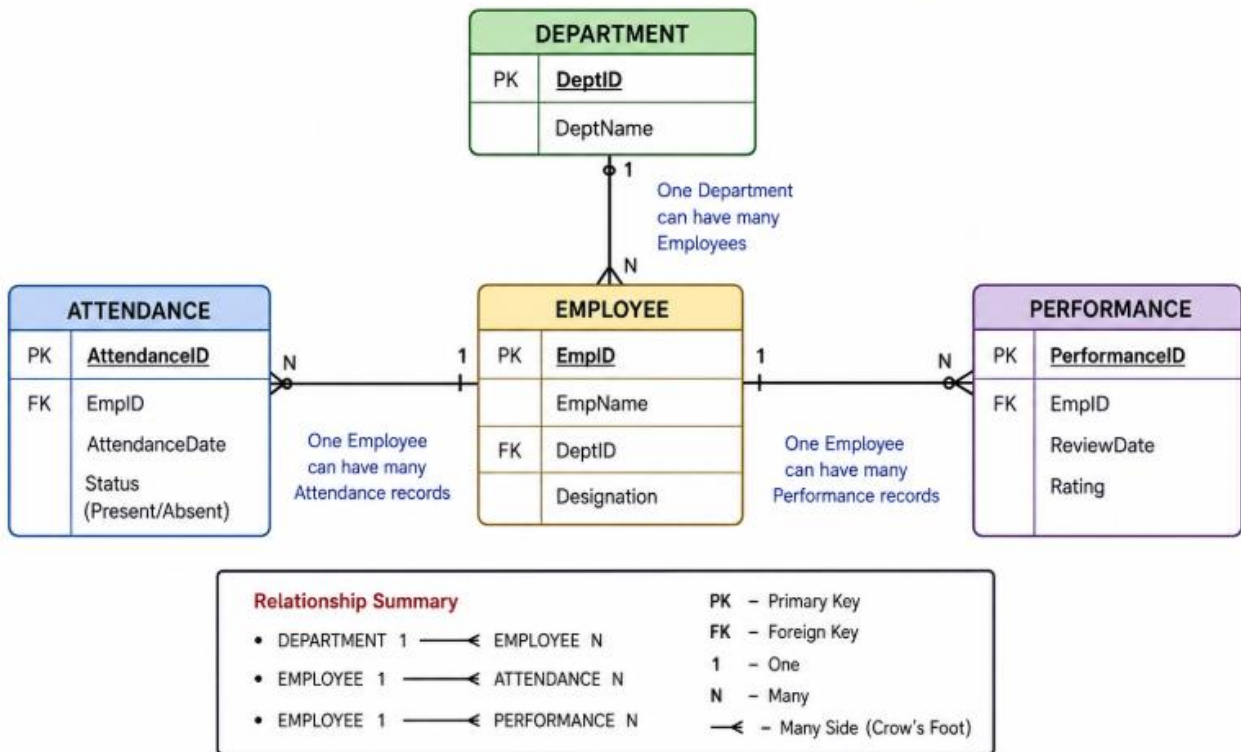
```

);
EMPLOYEE
CREATE TABLE EMPLOYEE
(
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    DeptID INT,
    Designation VARCHAR(50),
    FOREIGN KEY (DeptID)
        REFERENCES DEPARTMENT(DeptID)
);
ATTENDANCE
CREATE TABLE ATTENDANCE
(
    AttendanceID INT PRIMARY KEY,
    EmpID INT,
    AttendanceDate DATE,
    Status VARCHAR(10),
    FOREIGN KEY (EmpID)
        REFERENCES EMPLOYEE(EmpID)
);
PERFORMANCE
CREATE TABLE PERFORMANCE
(
    PerformanceID INT PRIMARY KEY,
    EmpID INT,
    ReviewDate DATE,
    Rating DECIMAL(3,2),
    FOREIGN KEY (EmpID)
        REFERENCES EMPLOYEE(EmpID)
);

```

3. Relationship Between Tables

HR SYSTEM – RELATION BETWEEN TABLES



4. Materialized View for Monthly Attendance

A **materialized view** stores the result of a query physically and can be refreshed when the underlying data changes.

We can create a monthly attendance summary containing:

- Employee ID
- Employee Name
- Month
- Total working days recorded
- Present days
- Absent days

SQL

```
CREATE MATERIALIZED VIEW MonthlyAttendanceSummary AS SELECT
    E.EmpID,
    E.EmpName,
    EXTRACT(YEAR FROM A.AttendanceDate) AS Year,
    EXTRACT(MONTH FROM A.AttendanceDate) AS Month,
    COUNT(*) AS TotalDays,
    SUM(CASE WHEN A.Status = 'Present' THEN 1 ELSE 0 END) AS PresentDays,
    SUM(CASE WHEN A.Status = 'Absent' THEN 1 ELSE 0 END) AS AbsentDays
FROM EMPLOYEE E JOIN ATTENDANCE A ON E.EmpID = A.EmpID
GROUP BY
    E.EmpID,
    E.EmpName,
    EXTRACT(YEAR FROM A.AttendanceDate),
    EXTRACT(MONTH FROM A.AttendanceDate);
```

5. Query the Attendance Materialized View

```
SELECT *FROM MonthlyAttendanceSummary;
```

This displays the monthly attendance summary of employees.

6. Materialized View for Monthly Performance

Now create a materialized view to calculate the **average performance rating** of each employee for each month.

```
CREATE MATERIALIZED VIEW MonthlyPerformanceSummary ASSELECT
  E.EmpID,
  E.EmpName,
  EXTRACT(YEAR FROM P.ReviewDate) AS Year,
  EXTRACT(MONTH FROM P.ReviewDate) AS Month,
  AVG(P.Rating) AS AverageRatingFROM EMPLOYEE EJOIN PERFORMANCE PON
E.EmpID = P.EmpIDGROUP BY
  E.EmpID,
  E.EmpName,
  EXTRACT(YEAR FROM P.ReviewDate),
  EXTRACT(MONTH FROM P.ReviewDate);
```

7. Query the Performance Materialized View

```
SELECT *FROM MonthlyPerformanceSummary;
```

This displays the monthly average performance rating of employees.

8. Refresh Materialized Views

When the underlying data changes, the materialized views can be refreshed.

```
REFRESH MATERIALIZED VIEW MonthlyAttendanceSummary;
```

```
REFRESH MATERIALIZED VIEW MonthlyPerformanceSummary;
```